A service's 'STATUS' can be 'OK', 'WARNING', 'CRITICAL', or 'UNKNOWN', which entirely depends upon the corresponding threshold values defined in the script.

Exit status 0 denotes an 'OK' status for the service, which is responsible for highlighting the service check with green on the Nagios Web console. Similarly, *exit status 1* denotes the 'WARNING' status of the service highlighted with yellow; *exit status 2* denotes the 'CRITICAL' service status highlighted with red; and *exit status 3* denotes the 'UNKNOWN' service status highlighted with grey.

Consider that we have to create a custom Nagios plugin to monitor the load average on a Linux server. For this, we need to decide the WARNING and CRITICAL thresholds, which are usually 80 per cent and 90 per cent, respectively, of the number of processor cores available in the system.

The live status of the load average can be obtained from the 'uptime' command, which can then be compared with the threshold levels, to decide the status of the service. Thresholds can be set using the */proc/cpuinfo* file, which possesses the necessary information related to the processor installed in the system. There are some conditional statements to compare live values with the threshold values. The echo statements display the service check in the proper format, and a few exit commands help to colour the service check on the Nagios Web console, accordingly.

The sample Bash script will look like what's shown below:

```
#!/bin/bash

UPTIME=`uptime | awk -F "average:" '{print $2}'`
load1=`echo $UPTIME | awk -F, '{print $1}' | xargs`
load5=`echo $UPTIME | awk -F, '{print $2}' | xargs`
load15=`echo $UPTIME | awk -F, '{print $3}' | xargs`

intload1=`echo "scale=1; $load1*100" | bc -l | cut -d "." -f 1`
intload5=`echo "scale=1; $load5*100" | bc -l | cut -d "." -f 1`
intload15=`echo "scale=1; $load15*100" | bc -l | cut -d "." -f 1`

Nprocs=`grep "processor" /proc/cpuinfo | wc -l`

warn=`scale=1; echo "80 * $Nprocs" | bc -l | cut -d "." -f 1`
crit=`scale=1; echo "90 * $Nprocs" | bc -l | cut -d "." -f 1`

actwarn=`echo "scale=1; $warn/100" | bc -l`
actcrit=`echo "scale=1; $crit/100" | bc -l`

if [ "$intload1" -le "$warn" -a "$intload5" -le "$warn" -a
"$intload15" -le "$warn" ]
then
        STATUS="OK"
        EXIT="0"
elif [ "$intload1" -gt "$warn" -o "$intload5" -gt "$warn" -o
"$intload15" -gt "$warn" ]
then
        if [ "$intload1" -gt "$crit" -o "$intload5" -gt
"$crit" -o "$intload15" -gt "$crit" ]
        then
        STATUS="CRITICAL"
        EXIT="2"
        else
        STATUS="WARNING"
        EXIT="1"
        fi
else
STATUS="UNKNOWN"
EXIT="3"
fi

echo "$STATUS- Load Average: $load1, $load5, $load15 |
load1=$load1;$actwarn;$actcrit;; load5=$load5;$actwarn;$actc
rit;;
load15=$load15;$actwarn;$actcrit;;" && exit $EXIT
```

When the script is created, it has to be put in the */usr/local/nagios/libexec* directory of the Nagios server as well as the clients to be monitored.

To create a command using the custom script, the *commands.cfg* file needs to be edited.

```
$ vi /usr/local/nagios/etc/objects/commands.cfg
define command{
        command_name check_load_average
        command_line $USER1$/check_nrpe -H $HOSTADDRESS$ -c
load_average
        }
```

Now, add the above command to the remote host's service configuration file.

```
$ vi /usr/local/nagios/etc/objects/services/MyLinuxServer.cfg
define service {
        use                             generic-service
        host_name                       MyLinuxServer
        service_description             Load Average
        check_command                   check_load_average
        }
```

Restart the Nagios service and perform a check from the command line:

```
$ /usr/local/nagios/libexec/check_nrpe -H 192.168.0.108 -c
load_average
```

Your custom Nagios plugin has been created. END

By Mandar Shinde

The author works in the IT division of one of the largest commercial automotive organisations in India. His technical interests include Linux, networking, backups and virtualisation.